

UPC

ControlBet

Startup

Integrantes:

- Yeissen Macalupu Marchan
- Fabricio Rivera Rupay
- Paula Chavez Pino
- Brayan Huerta Cardenas
- Giovanni Gallegos De La Cruz



Descripcion del problema

Problema

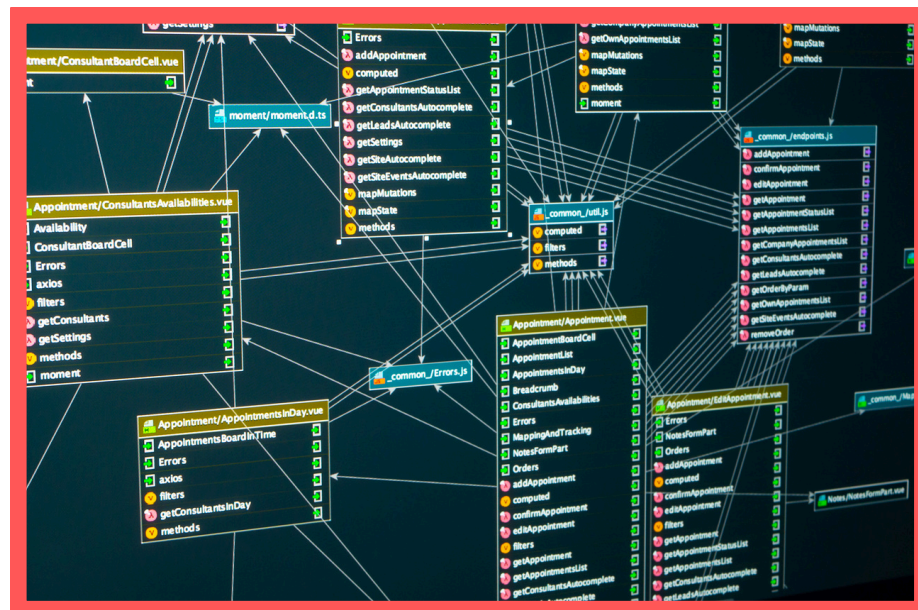
Información dispersa y poco estructurada en casas de apuestas.

Impacto

Errores en registros, demoras en validaciones y riesgo de pérdidas monetarias.

Necesidad

Centralizar usuarios, apuestas, cuotas, pagos y resultados en una base de datos.



Solución: BetCoreDB

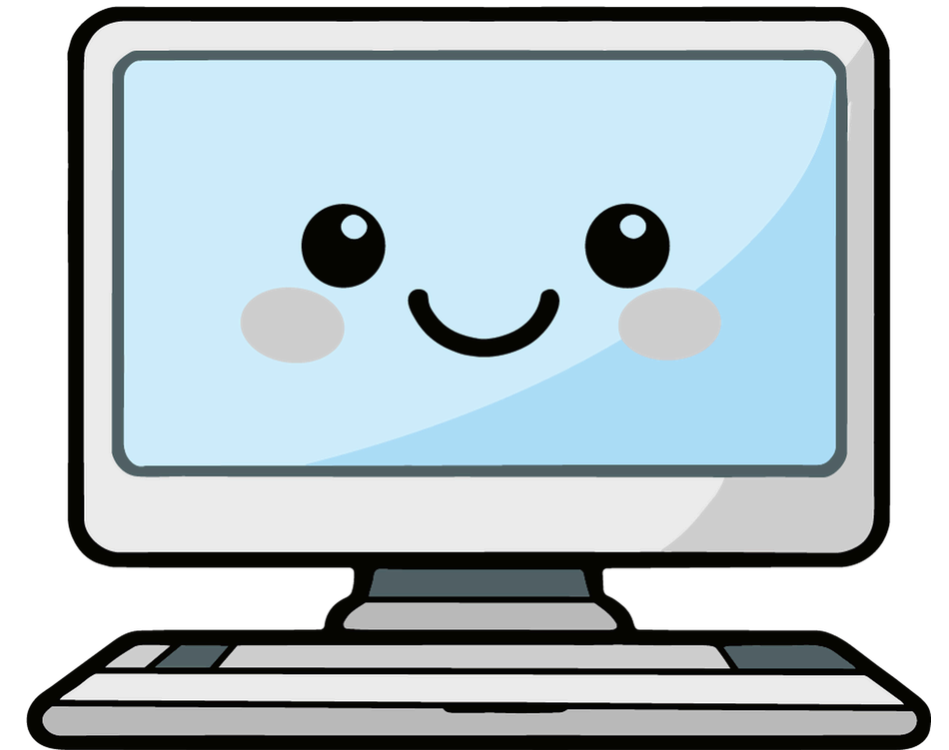
BetCore DB propone una base de datos relacional para organizar la información de usuarios, eventos deportivos, apuestas, cuotas, transacciones y comprobantes.

Ventajas de la solución

1. Centraliza la información del negocio.
2. Mejora el control de apuestas y pagos.
3. Reduce errores operativos.
4. Permite trazabilidad de cada operación.
5. Facilita consultas y reportes.



Alcance del sistema de base de datos

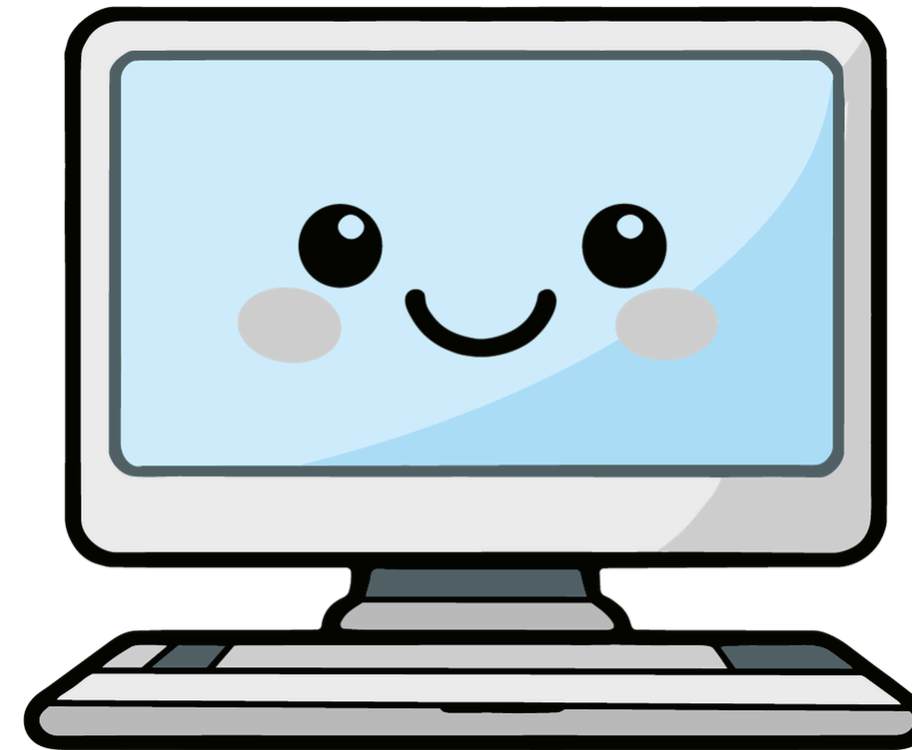
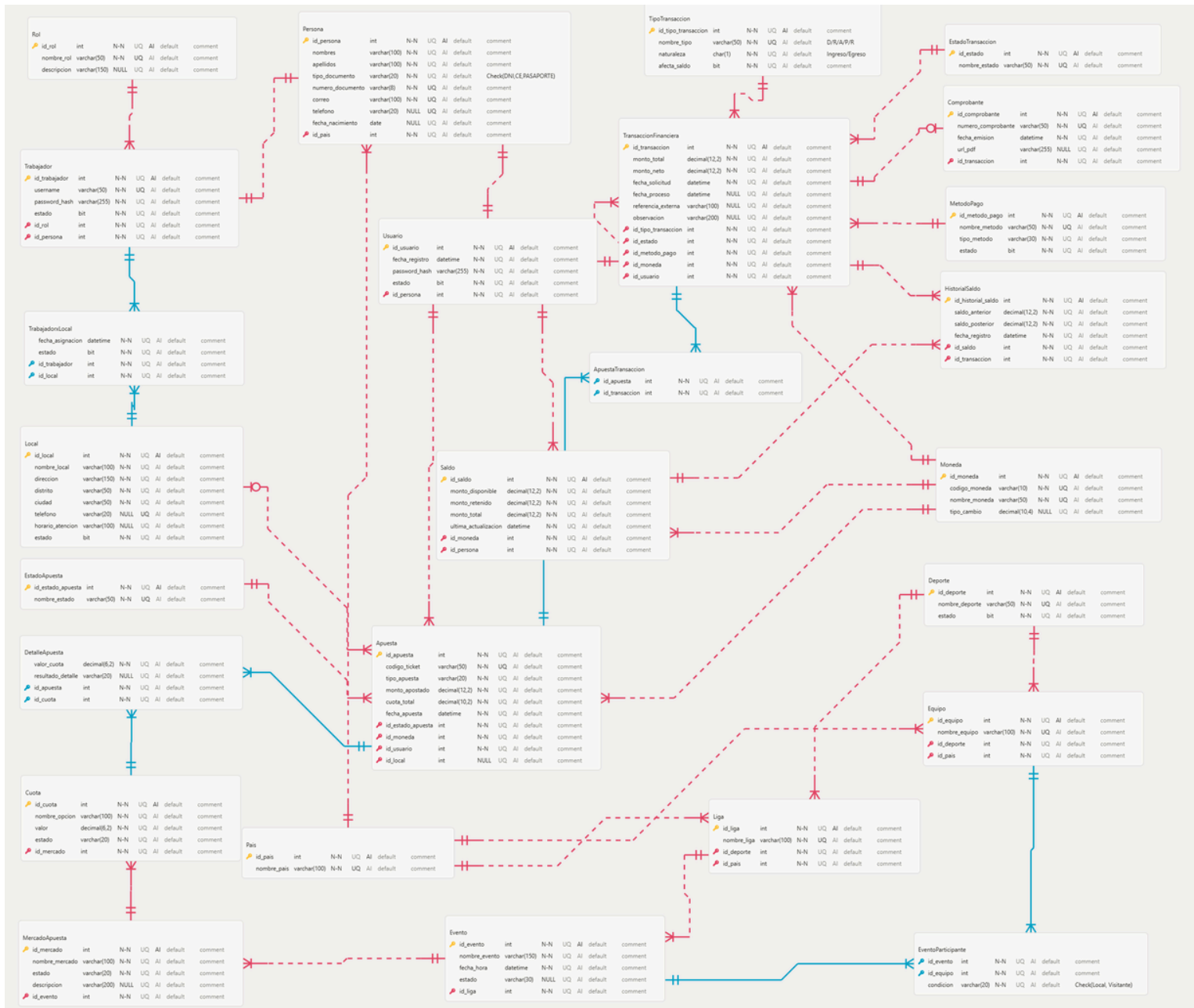


Elección del motor de base de datos SQL

¿Por qué usamos SQL Server?

Criterio	MySQL	PostgreSQL	SQL Server
Escalabilidad	Buena para apps web	Buena para modelos complejos	Adecuada para entornos transaccionales
Seguridad	Usuarios y privilegios	Roles y permisos	Permisos, auditoría, respaldo y recuperación
Compatibilidad	Requiere adaptar sintaxis	Requiere ajustes	Compatible con nuestro script
Facilidad de uso	Necesita adaptación	Necesita adaptación	Ejecutable en SQL Server Management Studio

Diagrama entidad- relación físico



Elección del motor de base de datos NoSQL

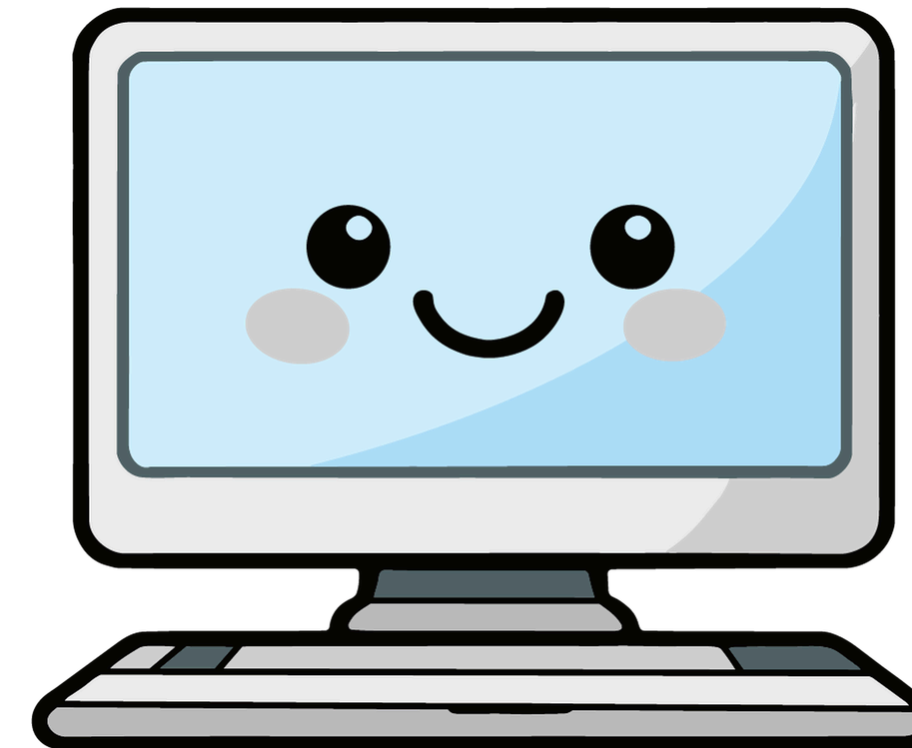
¿Por qué usamos MongoDB?

Tipo de base de datos NoSQL	Clave-valor	Familia de columnas	Grafos	Documental
Descripción	Organiza los datos mediante pares de clave y valor. Es útil para accesos rápidos a datos simples.	Organiza los datos en columnas agrupadas, útil para grandes volúmenes analíticos.	Representa datos mediante nodos y relaciones. Es útil cuando las conexiones entre entidades son el elemento principal.	Organiza la información en documentos flexibles, similares a JSON, permitiendo documentos embebidos y arreglos.
Evaluación para BetCore DB	No se considera la mejor opción, porque el proyecto requiere representar documentos con estructuras internas como usuario, local, moneda y selecciones.	No es la opción más adecuada, ya que el proyecto necesita manejar documentos con datos relacionados y no únicamente columnas distribuidas.	No es la prioridad del proyecto, porque BetCore DB se enfoca más en registrar operaciones, apuestas y movimientos financieros que en analizar redes complejas de relaciones.	Es la opción más adecuada, porque permite modelar apuestas y movimientos financieros de forma clara, flexible y cercana a las consultas del negocio.

Diagrama en Hackolade

apuestas		
_id	str	*
id_apuesta	int	*
codigo_ticket	str	*
usuario	obj	*
id_usuario	int	*
nombre_completo	str	*
tipo_documento	str	*
numero_documento	str	*
correo	email	
local	obj	
id_local	int	
nombre_local	str	
distrito	str	
ciudad	str	
selecciones	arr	*
[0]	obj	
deporte	str	*
evento	str	*
mercado	str	*
opcion	str	*
cuota	num	*
resultado	str	*
moneda	obj	*
id_moneda	int	*
codigo	str	*
simbolo	str	
monto_apostado	num	*
monto_ganancia	num	
estado_apuesta	str	*
fecha_apuesta	date-time	*
created_at	date-time	
updated_at	date-time	
fuelle	str	

movimientos_financieros		
_id	str	*
id_transaccion	int	*
usuario_id	int	*
apuesta_id	int	*
moneda	obj	*
codigo	str	*
simbolo	str	
tipo	str	*
estado	str	*
monto	num	*
saldo_anterior	num	
saldo_posterior	num	
comision_aplicada	num	
metodo_pago	str	
numero_comprobante	str	
descripcion	str	
fecha_operacion	date-time	*
created_at	date-time	
updated_at	date-time	



Principales consultas



```
// Consulta 1: Listar apuestas pendientes
db.apuestas
  .find(
    {
      estado_apuesta: "PENDIENTE",
    },
    {
      _id: 0,
      id_apuesta: 1,
      codigo_ticket: 1,
      "usuario.nombre_completo": 1,
      monto_apostado: 1,
      estado_apuesta: 1,
      fecha_apuesta: 1,
    },
  )
  .sort({
    fecha_apuesta: -1,
  });
```

Esta consulta muestra las apuestas que actualmente se encuentran con estado PENDIENTE, incluyendo el código de ticket, usuario, monto apostado y fecha de registro.

Resultado

```
√{
  id_apuesta: NumberInt('1019'),
  codigo_ticket: 'APU2024070019',
  > usuario: { ...
  },
  monto_apostado: NumberInt('100'),
  estado_apuesta: 'PENDIENTE',
  fecha_apuesta: ISODate
    ('2024-07-19T18:30:00.000Z')
}
```

```
√{
  id_apuesta: NumberInt('1018'),
  codigo_ticket: 'APU2024070018',
  > usuario: { ...
  },
  monto_apostado: NumberInt('55'),
  estado_apuesta: 'PENDIENTE',
  fecha_apuesta: ISODate
    ('2024-07-18T17:30:00.000Z')
}
```

Finalidad

Permite identificar las apuestas que aún no han sido resueltas. Esto es útil para el seguimiento operativo, ya que ayuda a controlar qué apuestas todavía no han sido ganadas, perdidas, anuladas o canceladas.

Principales consultas



```
// Consulta 3: Consultar el historial de apuestas de un usuario
db.apuestas
.find(
  {
    "usuario.numero_documento": "12345678",
  },
  {
    _id: 0,
    id_apuesta: 1,
    codigo_ticket: 1,
    "usuario.nombre_completo": 1,
    selecciones: 1,
    monto_apostado: 1,
    monto_ganancia: 1,
    estado_apuesta: 1,
    fecha_apuesta: 1,
  },
)
.sort({
  fecha_apuesta: -1,
});
```

Esta consulta permite buscar las apuestas registradas por un usuario específico mediante su número de documento.

Finalidad

Es útil para atención al cliente, auditoría interna y seguimiento del comportamiento de los apostadores, ya que permite revisar el historial de apuestas de un usuario determinado.

Resultado

```
{
  id_apuesta: NumberInt('1001'),
  codigo_ticket: 'APU2024070001',
  usuario: {
    nombre_completo: 'Juan Carlos Mendoza Peña'
  },
  selecciones: [
    {
      deporte: 'Fútbol',
      evento: 'Perú vs Chile - Eliminatorias 2026',
      mercado: 'Ganador del Partido',
      opcion: 'Perú',
      cuota: Double('2.45'),
      resultado: 'PENDIENTE'
    },
    {
      deporte: 'Tenis',
      evento: 'Wimbledon - Final Masculina',
      mercado: 'Ganador',
      opcion: 'Jannik Sinner',
      cuota: Double('1.95'),
      resultado: 'ACERTADO'
    },
    {
      deporte: 'Baloncesto',
      evento: 'NBA - Lakers vs Celtics',
      mercado: 'Over/Under Total Puntos',
      opcion: 'Over 215',
      cuota: Double('1.8'),
      resultado: 'PENDIENTE'
    }
  ],
  monto_apostado: Double('150.5'),
  monto_ganancia: Double('387.45'),
  estado_apuesta: 'PENDIENTE',
  fecha_apuesta: ISODate('2024-07-01T10:30:00.000Z')
}
```

Principales consultas



```
// Consulta 9: Consultar movimientos financieros de una apuesta específica
db.movimientos_financieros
.find(
  {
    apuesta_id: 1001,
  },
  {
    _id: 0,
    id_transaccion: 1,
    apuesta_id: 1,
    usuario_id: 1,
    tipo: 1,
    estado: 1,
    monto: 1,
    saldo_anterior: 1,
    saldo_posterior: 1,
    fecha_operacion: 1,
  },
)
.sort({
  fecha_operacion: -1,
});
```

Esta consulta busca los movimientos financieros asociados a una apuesta específica mediante el campo `apuesta_id`.

Finalidad

Es útil para auditoría, atención de reclamos y validación de operaciones económicas relacionadas con una apuesta determinada, como pagos, premios o reembolsos.

Resultado

```
▼ {
  id_transaccion: NumberInt('2001'),
  usuario_id: NumberInt('501'),
  apuesta_id: NumberInt('1001'),
  tipo: 'APUESTA',
  estado: 'COMPLETADA',
  monto: Double('150.5'),
  saldo_anterior: NumberInt('500'),
  saldo_posterior: Double('349.5'),
  fecha_operacion: ISODate('2024-07-01T11:15:00.000Z')
}
```

Muchas
gracias

